

■ CRA TEAM 3 · TEAM CTRL-C

CodeFab

새로운 확장자 `.ctrlc`로 코드를 분석하고 실행하는 Interpreter



목차

01

팀 소개

역할과 구현 분담

02

프로젝트 목표

새로운 확장자 `.ctrlc`

03

전체 구조

Shell과 interpreter pipeline

04

주요 기능

Tokenizer · AST · Runtime

05

디자인 패턴

AST · Callable · Command

06

협업과 품질

Review · TDD · Actions

팀 역할과 구현 분담

김명준 팀장

architecture · CI

권소영

assembler · executor

김민준

expression · shell

박진우

tokenizer · array

이예진

checker · optimizer

`.ctrlc` 실행 목표

입력

- `.ctrlc` source
- prompt 한 줄 입력
- debug 실행 대상



결과

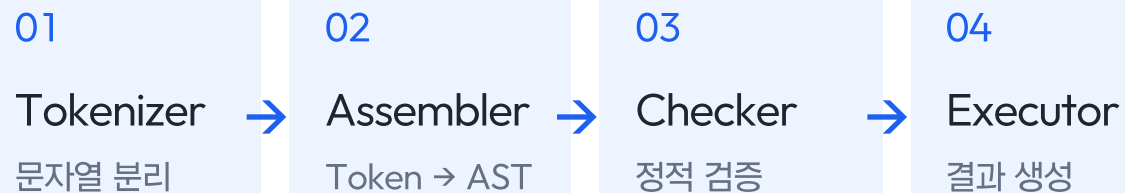
- Token / Expr / Stmt 구조화
- Checker 사전 검증
- Executor 실행 결과 생성

Shell과 Pipeline 구조

Shell Modes

- 01 Prompt Mode
한 줄 입력을 즉시 실행
- 02 File Mode
.ctrlc 파일 전체 실행
- 03 Debug Mode
Stmt 단위로 실행 상태 확인

Interpreter Pipeline



01

■ FEATURE IMPLEMENTATION

기능 구현 설명

입력 문자열이 Token, AST, 검증 단계를 지나 실제 실행 결과가 되기까지의 핵심 구현을 차례로 봅니다.

Tokenizer가 문자열을 의미있는 Token 단위로 분리합니다

Tokenizer

소스 코드 문자열을 한 글자씩 읽으면서 예약어, 식별자, 연산자, 리터럴 단위의 token으로 분리합니다.

01 공백 스킵

공백은 건너뛰고, 개행은 Token의 line 위치값을 증가합니다.

02 예약어 · 식별자 구분

예약어에 있는지, identifier로 정의 가능한 형태인지 등을 확인합니다.

03 Token 저장

TokenType과 실제 값(origin)을 담은 Token 리스트를 Assembler로 전달합니다.

var " name " = " "ctrlc" ;

↓ 한 글자씩 확인하며, 타입에 따라 분기

var 문자로 시작 → 예약어 테이블 조회 → 일치

name 문자로 시작 → 예약어 테이블 조회 → 없음

= 기호로 시작 → 등록된 토큰값과 일치

"ctrlc" "로 시작 → 닫는 "가 나올 때까지 읽어 하나로 묶음

; 기호로 시작 → 등록된 토큰값과 일치

↓ Token(type, origin)으로 저장

VAR
var

IDENTIFIER
name

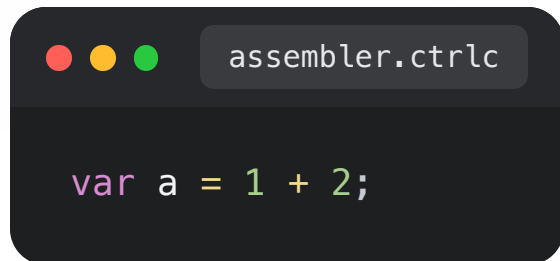
EQUAL
=

STRING
"ctrlc"

SEMICOLON
;

이 문장에서 나온 Token은 5개 · 전체 TokenType은 총 44개 정의되어 있습니다.

Assembler는 Token을 실행 가능한 AST로 조립합니다



```
assembler.ctrlc  
  
var a = 1 + 2;
```

생성된 AST

VarStmt

```
└─ BinaryExpr(+)  
   └─ LiteralExpr(1)  
      └─ LiteralExpr(2)
```

Assembler는 실행하지 않고
AST만 생성합니다.

Expression

값을 계산하는 node

Literal · Binary · Variable · Call · Get

Statement

실행 흐름을 표현하는 node

Var · Print · Block · If · For · Func · Class · Import

Environment는 식별자와 Scope Chain을 관리합니다

Global Environment

변수

x = 10

Native Function

print · Array

User Function

closure

Class

method table

Instance

field · method

Import

alias · member

Function 호출 시 Scope Chain

Function Environment

a = 2

b = 3

local variables

parent
→

Global Environment

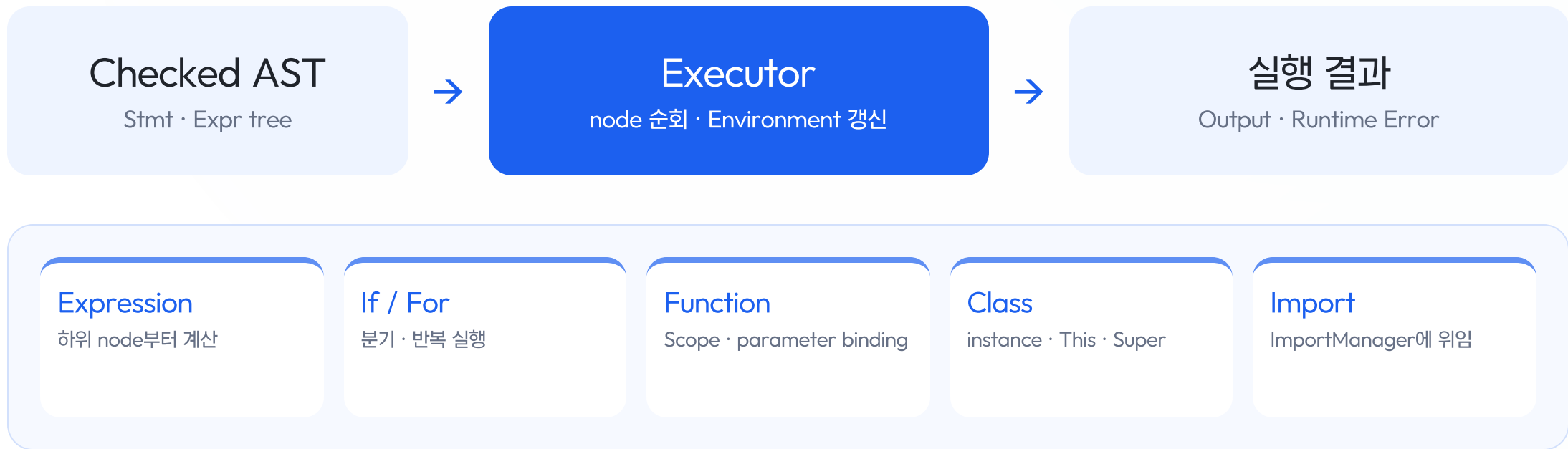
functions

classes

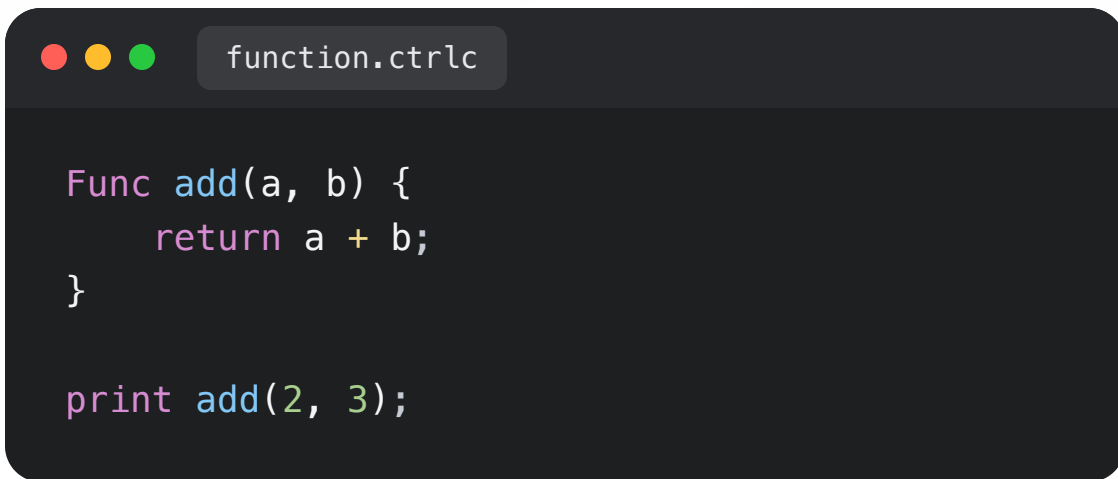
imports

현재 Scope에 이름이 없으면 parent를 따라 Global까지 탐색합니다.

Executor는 AST를 순회하며 실제 프로그램을 실행합니다



Function은 새 Environment에서 body를 실행합니다



```
Func add(a, b) {  
    return a + b;  
}  
  
print add(2, 3);
```

호출 흐름

선언 시 함수 객체를 저장하고, 호출 시 새 Environment에서 body를 실행합니다.

- 01 선언
FunctionStmt → UserFunction
- 02 호출
parameter와 argument binding
- 03 반환
return 값을 호출 지점으로 전달

Class는 method table과 This binding으로 동작합니다

객체 실행 흐름

class 생성과 method binding을 Callable 흐름으로 처리합니다.

- 01 UserClass
method table
- 02 UserInstance
field / method lookup
- 03 This binding
instance 연결
- 04 Super lookup
parent method 탐색

```
class.ctrlc

Class Robot {
    move() {
        print "move";
    }
}

Class SpeedRobot : Robot {
    move() {
        Super.move();
        print "speed";
    }
}

var r = SpeedRobot();
r.move();
```

Array() 로 배열을 생성할 수 있습니다

생성과 접근

Function과 Class가 쓰던 Callable을 그대로 이용해, 새 문법 없이 Array를 구현했습니다.

01 Array() 호출

NativeFunction으로 등록된 Array가 Callable 규칙에 따라 실행됩니다

02 크기 검사 후 생성

size가 정수이고 0 이상인지 확인한 뒤 `[None] * size`로 초기화합니다

03 배열 사용

`a[i]` 형태로 사용하며, 위치에 따라 읽기 또는 쓰기로 구분합니다

04 실행 중 검증

index가 정수인지, 0 이상 길이 미만인지 Executor가 확인합니다

```
array.ctrlc  
  
var arr = Array(3);  
arr[0] = "first";  
  
print arr[0];
```

실행 중 **Import**를 만나면 다시 Pipeline을 수행합니다

왜 ImportStmt를 별도로 두었나?

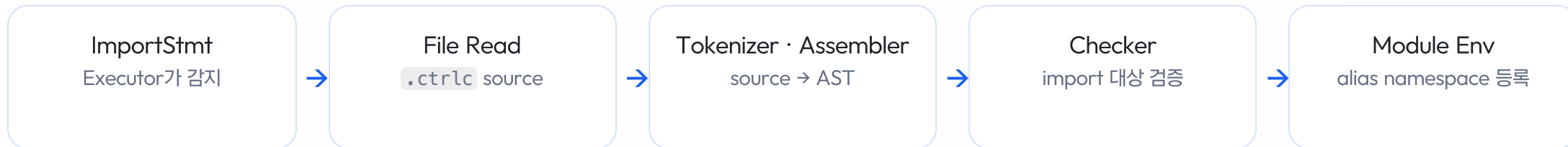
Executor가 import를 감지하더라도, 대상 파일은 다시 Tokenize · Assemble · Check를 거친 뒤 module namespace로 등록되어야 합니다. Executor 내부에 모두 구현하면 Executor의 책임이 지나치게 커지게 됩니다.

Executor

ImportStmt 실행 · module environment 연결

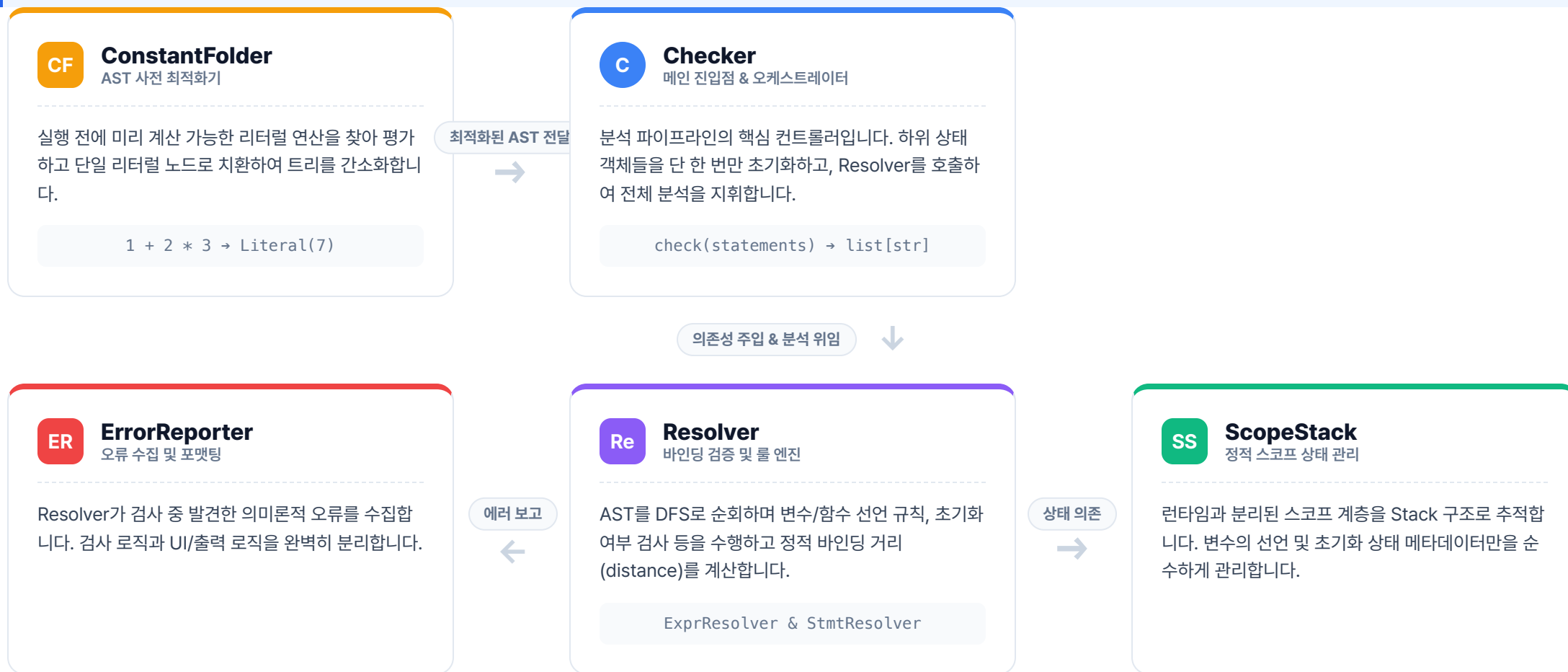
ImportManager

경로 해석 · cache · 순환 추적



Checker 모듈은 5개의 독립된 하위 모듈로 협력합니다

Checker 전반적인 기능 및 역할: 파서가 생성한 AST(추상 구문 트리)를 런타임 실행 전에 순회하며 정적 의미 분석(Static Semantic Analysis)을 수행합니다. 실행 전 계산 가능한 식을 최적화하고, 변수/함수의 선언 및 스코프 참조 무결성을 검증하여 런타임 오류를 사전에 차단합니다.



유연한 의존성 주입(DI)과 결합도를 낮춘 클린 아키텍처

Design Philosophy

Checker 모듈은 단일 책임 원칙(SRP)을 준수하며, 안전한 검증과 유연한 기능 확장을 위해 객체 간의 결합도를 획기적으로 낮추었습니다.

01 의존성 주입 (DI)

내부에서 상태를 직접 생성하지 않고 외부에서 1회 생성 후 연쇄 주입하여, 전역 상태 오염을 방지합니다.

02 단일 책임 원칙 (SRP)

상태 저장(ScopeStack), 에러 출력(ErrorReporter), 규칙 판단(Resolver)을 엄격하게 분리하여 관리합니다.

03 테스트 용이성 확보

관심사가 분리된 구조 덕분에 Test Double(Mock) 객체를 손쉽게 교체 주입하여 독립적인 TDD 환경을 구축합니다.

DI 의존성 주입 아키텍처 흐름



CLEAN 클린 코드 및 설계 원칙

SRP 상태와 판단 로직의 완벽한 분리

ScopeStack 은 순수 메타데이터 관리(저장/복원)만 수행하고, **Resolver** 는 바인딩 조건 체크 등 비즈니스 룰만 처리하도록 관심사를 철저히 분리했습니다.

OCP Dispatch Table 기반 매핑 적용

노드 검사 시 길어지는 `if/elif` 사슬을 배제하고, `type(node) -> handler` 딕셔너리를 채택해 신규 문법 추가 시 로직 수정 없이 테이블 등록만으로 확장 가능합니다.

Testability 독립적인 단위 테스트(TDD) 보장

내부에서 인스턴스를 직접 만들지 않으므로, 테스트 환경에서 **ErrorReporter** 등을 Dummy/Mock 객체로 쉽게 교체하여 순수 로직만 고립시켜 검증할 수 있습니다.

Shell은 실행 모드를 고르고 Pipeline으로 위임합니다

실행 모드

- 01 Prompt Mode
한 줄 입력을 즉시 실행
- 02 File Mode
.ctrlc 파일 전체 실행
- 03 Debug Mode
breakpoint, step, watch로 상태 확인

Shell 책임

입력 분류 · command / code 판단

상태 관리 · history / debug context

Factory → Command.execute(shell)

Runner → 공통 Interpreter Pipeline

Custom Language Features

확장 방향

팀 전용 문법도 같은 pipeline에서 처리되도록 확장했습니다.

한글 Alias 적용

Token에 별칭 적용

ctrl_c()

팀 소개 native function



연산

이름 공합 점수 계산

.ctrlc

파일 확장자

Alias 예시

Alias 적용 전

```
battle_game.ctrlc

for (
  var round = 1;
  hero.isAlive() and slime.isAlive();
  round = round + 1
) {
  if (round == 2) {
    hero.ultimate(slime);
  } else {
    hero.attack(slime);
  }
  if (slime.isAlive()) {
    slime.attack(hero);
  }
}
```

Alias 적용 후

```
battle_game.ctrlc

또또 (
  값 라운드 = 1;
  히어로.생존여부() 이랑 슬라임.생존여부();
  라운드 = 라운드 + 1
) {
  만약 (라운드 같다 2) {
    히어로.필살기(슬라임);
  } 아니면 {
    히어로.공격(슬라임);
  }
  만약 (슬라임.생존여부()) {
    슬라임.공격(히어로);
  }
}
```

02

■ DESIGN PATTERNS

디자인 패턴

복잡해지는 구현 지점을 패턴으로 어떻게 정리했는지 봅니다.

디자인 패턴 적용 사례

Composite

AST node 구성

구현: Stmt · Expr tree

효과: Checker와 Executor가 같은 구조 순회

Adapter

Callable 호출 계약

구현: Native · Function · Class call 통합

효과: Executor는 call()만 호출

Command

Shell 명령 실행

구현: Exit · Recommend · RunCode 분리

효과: PromptShell의 분기 축소

Composite는 중첩 문법을 하나의 AST tree로 표현합니다

문제

for, if, block이 서로 중첩됩니다

문법마다 별도 실행 구조를 만들면 검증과 실행 코드가 쉽게 갈라집니다.

패턴 적용

Stmt와 Expr를 tree node로 조립

- ForStmt 안에 VarStmt
- 조건은 BinaryExpr
- 본문은 BlockStmt

결과

Checker와 Executor가 같은 AST를 순회

문법이 추가되어도 node 단위로 확장하고, pipeline 전체 구조는 유지됩니다.

Callable Adapter는 서로 다른 호출 대상을 같은 방식으로 실행합니다

문제

함수, Native, Class 생성은 내부 동작이 다릅니다

Executor가 대상별 세부 구현을 모두 알면 호출 분기가 계속 늘어납니다.

패턴 적용

모두 Callable 계약으로 맞추

- arity()로 인자 개수 확인
- call(executor, args)로 실행
- UserClass 생성도 같은 call

결과

Executor는 호출 규칙 하나만 유지

새 호출 대상을 추가해도 Executor의 핵심 흐름을 크게 바꾸지 않습니다.

Command Pattern은 Shell 입력 분기를 명령 객체로 분리합니다

문제

PromptShell이 입력 종류와 실행을 함께 처리

exit, ctrlc, ctrlv, explain 같은 명령이 늘수록 if 분기와 테스트 범위가 커졌습니다.

패턴 적용

Factory가 Command를 고르고 Command가 실행

- ShellCommandFactory.create()
- should_save_history()
- execute(shell)

결과

Shell은 선택과 위임에 집중

새 명령은 Command 클래스로 추가하고, history 정책도 명령별로 캡슐화합니다.

03

■ COLLABORATION

협업과 품질 기준

리뷰, 자동화, 보호 규칙으로 main 반영 기준을 맞췄습니다.

Code Review 사례 – Tokenizer 리팩토링 실패



✓

j0shuajun approved these changes 3 days ago

View reviewed changes

j0shuajun left a comment • edited ▾

Owner ...

현재 동작 기준으로는 문제 없어 보이고, TokenType 에서 실제 lexeme을 바로 볼 수 있어 관리 포인트가 줄어드는 점은 좋습니다.

다만 _SINGLE_CHARACTERS / _CHARACTERS_WITH_EQUAL / _RESERVED_WORDS 가 토큰의 "의미"가 아니라 문자열 형태(len , endswith , isalpha)로 그룹을 추론하고 있어서, 나중에 **, //, => 같은 다문자 연산자나 특수한 keyword가 추가되면 의도치 않게 누락될 수 있을 것 같습니다.

지금 규모에서는 이 PR 방향도 괜찮다고 보지만, 이 자동 분류 규칙이 tokenizer의 계약이라는 점을 짧게 주석으로 남기거나, 향후 토큰 추가 시 이 규칙을 같이 확인해야 한다는 테스트/문서 보강이 있으면 더 안심될 것 같습니다.



✓

re-nos approved these changes 3 days ago

View reviewed changes

re-nos left a comment

Collaborator ...

새롭게 추가되는 token의 분류 논리 조건이 복잡하다면 분류 카테고리가 계속 늘어나게 되진 않을까요?

Code Review 후 Main 미반영

TDD 기반 개발 후, TokenType에 따른 분기가 늘어나 자동 분류하도록 리팩토링 진행하였으나, 분류를 위한 암묵적인 규칙이 생기며 유지보수성이 떨어진다는 리뷰를 받아 close (Merge 불가)



Jinops commented 2 days ago • edited ▾

Collaborator Author ...

enum 값에 실제 매칭 텍스트를 직접 넣고, 용도별 분류(_SINGLE_CHARACTER_TOKENS 등) 디렉터리를 자동 그룹핑하도록 하여 이중 관리 및 휴먼 에러를 방지하고자 해당 PR을 작성하였습니다.

enum-텍스트 매칭은 해결이 되었습니다. 그러나 그룹핑의 경우, 디렉터리에 수동 작성하던 것을 암묵적 규칙으로 옮긴 것이라 신규 토큰 분류 시 누락 가능성이 해결되지 않았고, 다른 개발자가 보기에 암묵적 규칙을 한 번 더 이해해야 하는 부담이 생겼습니다. 또한 규칙으로 옮기는 과정에서 코드 가독성이 떨어졌고, 오히려 기존 방식이 더 직관적으로 보입니다.

@j0shuajun 님, @re-nos 님께서 위 부분을 잘 지적해주셨습니다.

따라서 해당 refactoring PR은 close하고, 기존 방식을 유지하다가 필요 시 개선된 PR을 올리겠습니다.

Code Review 사례 – Tokenizer 리팩토링 재시도



최소 변경점으로 Main 반영

Token 기능개발 고도화 후, 이전 PR의 자동분류는 over engineering으로 판단하여, 필요한 개발 범위에 맞게 리팩토링 후 PR 및 merge 완료

TDD와 회귀 테스트 사례

Red-Green 반복

PR #15·#18: [test] 커밋 다음 [feat] 커밋이
기능 단위로 이어짐

세분화된 test-first

토큰 하나 단위로 커밋을 쪼개 커밋 로그만으로 검증
대상 파악

리뷰 효율

17~19개 세분 커밋에도 가독성 좋다는 평가로
이어짐

- TEST add plus calculation test
- FEAT add plus tokenizer
- TEST add multiply operation test
- FEAT add multiply tokenizer
- TEST add minus and divide operation
- FEAT add minus and divide token
- TEST block scope / grouping / modify block scope
- FEAT grouping and block scope
- TEST less than, greater than
- FEAT less than, greater than token
- TEST comma, bang / fix assign_with_comma
- FEAT add comma, bang token

Before/After로 보는 Shell 구조 개선

Before

```
prompt_shell.py

if source in ("exit", "quit"):
    self.is_running = False
    return ["Bye!"]

elif source == "ctrlc":
    return self.recommend_command()

elif source == "ctrlv":
    return self.run_recommended_command()
```

문제: PromptShell이 명령 선택, 실행, history 정책까지 직접 처리해 분기가 커지고 명령별 테스트 범위가 넓어졌습니다.

After

```
shell_command.py

command = ShellCommandFactory.create(source)

if command.should_save_history():
    shell.save_history(command)

outputs = command.execute(shell)
return outputs
```

효과: PromptShell은 command 선택과 실행 위임만 담당하고, 새 명령은 독립 Command 단위로 추가할 수 있게 되었습니다.

PR Template과 Makefile로 협업 절차를 표준화했습니다

!경 요약

- something changed

!고 사항

- comment

!련 링크

- 이슈: .
- 참고: .

elf-Review Checklist

R 준비

- ☐ 설명이 명확하고 이해하기 쉬움
- ☐ 적절한 라벨을 달았나요?

!스트

- ☐ 테스트 코드 작성/수정됨 (필요한 경우)
- ☐ 테스트 코드는 `tests/` 디렉토리의 올바른 위치에 있음
- ☐ 로컬에서 테스트 완료
- ☐ 기존 기능 동작 확인

의: 이 PR이 merge되려면 최소 2명 이상의 Approve가 필요합니다. PR 생성자가 모든 코멘트를 반영한 후 직접 merge를 진행해주세요.

Makefile 명령 표준화

make install 개발 의존성 설치 기준 통일

make lint black · isort · ruff 실행 순서 통일

make test pytest 실행 방법을 팀 공통 명령으로 고정

make run file mode 실행 방식을 한 줄 명령으로 공유

make debug debug mode 진입 방법을 동일하게 사용

GitHub Actions로 Merge 전 품질 기준을 자동 확인했습니다

✓ All checks have passed
4 successful checks

✓	 codecov/patch — Coverage not affected when comparing dcfed36...58600b3	...
✓	 Gist Acceptance / gist acceptance (pull_request) Successful in 12s	Required ...
✓	 Lint / lint (pull_request) Successful in 15s	Required ...
✓	 Unit Tests / unit tests (pull_request) Successful in 25s	Required ...

Required Checks

Lint, Unit Tests, Gist Acceptance를 PR 필수 통과 조건으로 뒤 main 반영 전 기준을 맞췄습니다.

Coverage Signal

Codecov patch 결과를 함께 확인해 변경 범위가 테스트 관점에서 안전한지 봅니다.

Review 전에 자동 검증

사람 리뷰는 설계와 의도에 집중하고, 반복 검사는 Actions가 먼저 확인합니다.

우리의 한 줄 소감

김명준

직접 개발한 내용이 아닌 코드를 리뷰하는 것이 꽤나 노력이 필요하다는 것을 몸소 체감했어요. 디자인 패턴을 생각하면서 구현하는 것도 어려워 보이더라고요. 그래도 코드 리뷰와 평소 개발 습관이 얼마나 중요한 지 다시 생각해보는 좋은 계기가 되었습니다. 우리 팀원분들 고생하셨고 감사합니다!

권소영

개발하랴 리뷰하랴 바쁜 일정이었지만 모두들 한 마음 한 뜻으로 진행했던 터라 큰 이슈 없이 잘 마무리된 것 같습니다. 모두 고생 많으셨어요!!

김민준

여러명에서 개발을 한 경험이 거의 없었는데 이번 기회에 함께 개발하고 리뷰 하면서 많이 배울 수 있었습니다. 팀원분들 모두 고생 많으셨고 감사드립니다.

박진우

생각했던 것 보다 더 유익한 Code Review Agent 과정이었습니다. AI 시대에 맞게 나 자신도 발빠르게 변화해야 하는데, 이번 과정을 통해 그 흐름에 잘 따라갈 수 있었습니다. 그리고 평소에는 보기 힘든 타 부서의 동료분들과 협업하면서 많이 배우고 생각이 넓어질 수 있었습니다. 고생하셨습니다!

이예진

unit test를 계속 작성하고 관리하며 개발해 나가는 과정이 처음엔 번거로웠지만, 여러 사람과 협업하며 프로젝트를 진행할 때 기존 코드의 유효성을 위해 꼭 필요한 과정임을 알게 되었습니다. 리뷰를 주고 받는 과정에서 제가 생각지 못했던 여러 설계 관점을 배울 수 있었습니다. 모두 고생 많으셨습니다~